

EXPERIMENT NO. 01 -Software Requirement Specification document in IEEE Format for the Smart Connect

AIM: To document and define the functional and non-functional requirements of the Smart Connect system in accordance with IEEE SRS standards.

Theory :

A Software Requirements Specification (SRS) is a comprehensive, structured document defining the functional, non-functional, and interface requirements for a software system to be developed. It acts as a binding contract between customers and contractors, serving as a blueprint for developers and testers to ensure project alignment, reduce risks, and control costs.

Key Aspects of an SRS

- **Purpose:** Outlines *what* the software will do and how it should perform, rather than *how* to build it.
- **Core Components:** Includes functional requirements (system features), non-functional requirements (security, performance, usability), and external interface requirements.
- **Target Audience:** Serves stakeholders, including project managers, developers, testers, and customers.
- **Benefits:** Reduces development cost and time, improves quality, ensures clear communication, and aids in creating test plans.

Essential Characteristics of a Good SRS

- **Correct & Unambiguous:** Accurate and easily understood with consistent interpretation.
- **Complete & Verifiable:** Contains all necessary requirements and is testable.
- **Consistent & Traceable:** No conflicting requirements; each requirement is uniquely identified and traceable to business needs.
- **Modifiable & Ranked:** Structured for easy changes and ranked by importance/stability.

Table of Contents

Table of Contents.....	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope.....	1
1.5 References	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics.....	2
2.4 Operating Environment	3
2.5 Design and Implementation Constraints	3
2.6 User Documentation.....	3
2.7 Assumptions and Dependencies.....	3
3. External Interface Requirements	3
3.1 User Interfaces.....	3
3.2 Hardware Interfaces	3
3.3 Software Interfaces.....	3
3.4 Communications Interfaces.....	4
4. System Features.....	4
4.1 System Feature 1	4
4.2 System Feature 2 (and so on)	4-6
5. Other Nonfunctional Requirements	6
5.1 Performance Requirements	6
5.2 Safety Requirements	7
5.3 Security Requirements	7
5.4 Software Quality Attributes	7
5.5 Business Rules.....	8
6. Other Requirements.....	8
Appendix A: Glossary	8
Appendix B: Analysis Models	8
Appendix C: To Be Determined List.....	8

1. Introduction

1.1 Purpose

This paper lists what the Online Examination System (OES) needs to do. It covers the full app for running tests online – like creating exams, students taking them, checking scores, and reports. The app also helps admin check the teacher, student management, course progress of active ones and the overall academics of both the student and faculty. A person can also take courses themselves except the ones given by faculty. This document covers only the software functionality.

1.2 Document Conventions

The following conventions are used:

- **Bold** for important modules such as Dashboard, Quizzes, Attendance
- *Italic* for actions such as *start quiz*, *view results*
- Bullet points and tables for clarity
- Priority levels:
 - High – Core features
 - Medium – Supporting features
 - Low – Optional enhancements

Higher priority requirements override lower ones unless specified.

1.3 Intended Audience and Reading Suggestions

Who this is for:

- **Developers/Admin:** Should read all sections to implement dashboards, quizzes, and analytics.
- **Faculty:** Focus on system capabilities and reporting features.
- **Students:** Refer to login, course, and quiz functionalities.

1.4 Product Scope

The Online Examination and Learning Management System is a web-based application designed to digitize educational activities such as course management, online assessments, attendance monitoring, and result generation. The primary goal of the system is to reduce manual academic processes and provide real-time access to academic data. It offers centralized control for administrators, structured course access for students, and performance tracking tools for instructors. The system improves efficiency, transparency, and accuracy in managing academic operations.

1.5 References

NA

2. Overall Description

2.1 Product Perspective

- The Online Examination and Learning Management System is a web-based standalone application designed to integrate academic learning activities and online assessment into a single digital platform. It eliminates the need for manual record keeping and physical examinations by providing an automated online environment.
- The system consists of a centralized dashboard that acts as the control center for administrators and instructors. It displays real-time information such as total users, available courses, active quizzes, attendance status, and recent system activities.
- A user management component is integrated to control student and instructor accounts. Administrators can add, update, deactivate, and monitor user access, ensuring secure and organized platform usage.
- The course management module enables instructors to create, modify, and organize courses along with learning materials. Students can enroll in these courses and access educational content through their personal dashboards.
- The quiz and examination module allows instructors to create online assessments with multiple-choice questions. Students can attempt exams within defined time limits, and the system automatically evaluates responses.

2.2 Product Functions

The system provides secure login authentication for different user roles including administrators, instructors, and students. After login, users access role-specific dashboards displaying relevant academic information. Administrators manage users and system data, while instructors create courses, schedule quizzes, and record attendance. Students enroll in courses, participate in learning activities, and attempt online examinations. The system performs automatic evaluation of quiz responses and generates instant results. Attendance data is stored digitally and linked to academic performance. Analytical tools present graphical and numerical performance reports that support academic monitoring and decision making.

2.3 User Classes and Characteristics

The administrator is the highest-level user responsible for managing platform operations, user accounts, courses, and system settings. Administrators typically have moderate technical expertise and oversee data security and workflow organization. Students are end users who access learning materials, enroll in courses, attempt quizzes, and view results. They generally possess basic digital literacy and rely on the system for academic participation.

2.4 Operating Environment

The Online Examination and Learning Management System operates within a web-based environment accessible through standard internet browsers. It supports commonly used operating systems including Windows, macOS, and Linux. The system requires a reliable internet connection to ensure uninterrupted access to course content, examinations, and real-time data updates.

2.5 Design and Implementation Constraints

The system is constrained to operate within a web application framework using centralized database storage. Security features are limited to username and password authentication due to project scope restrictions. Advanced technologies such as biometric verification, artificial intelligence-based monitoring, and large-scale cloud deployment are not included. The system must be lightweight and cost-effective to support academic demonstration purposes. Development is limited by time, budget, and educational objectives rather than enterprise requirements.

2.6 User Documentation **N.A.**

2.7 Assumptions and Dependencies **N.A.**

3. External Interface Requirements

3.1 User Interfaces

- Roles: student, teacher, admin, parent; guided via route-based pages under pages.
- Layout: shared header, sidebar, content area via Layout.js, Header.js, Sidebar.js, and AuthLayout.js.
- Styling: Tailwind CSS configured in tailwind.config.js; global styles in index.css.
- Navigation: React Router pages such as Dashboard, Courses, Assignments, Quizzes, Messages, Announcements, Gamification, Analytics, Profile, and Auth.
- Common UI components: buttons/inputs/cards/spinner in UI.
- Notifications & errors: toast-based feedback via react-hot-toast wired in api.js and real-time toasts in SocketContext.js.
- Validation: client-side form validation complemented by server-side rules in validation.js.

3.2 Hardware Interfaces **N.A.**

3.3 Software Interfaces

- Runtime & OS: Node.js Express backend (server.js); React frontend (src). OS-agnostic; Windows dev environment confirmed; deployable on Linux/macOS/Windows.

- Auth: JWT-based auth per auth.js and auth.js; token in Authorization: Bearer <jwt>.
- HTTP API: REST endpoints mounted under /api/* in server.js with feature-specific routers in routes.
- Configuration: environment variables MONGODB_URI, JWT_SECRET, PORT, NODE_ENV, CLIENT_URL, REACT_APP_API_URL, REACT_APP_SOCKET_URL, HUGGINGFACE_API_KEY.

3.4 Communications Interfaces

N.A.

4. System Features

4.1 Authentication & Roles

Description and Priority: Secure login/register, JWT auth, role-based access (High).

Stimulus/Response Sequences:

User submits credentials → server validates → returns JWT + profile.

Accessing protected route → token verified → data returned or 401.

Functional Requirements:

REQ-AUTH-1: Validate login and registration per validation.js.

REQ-AUTH-2: Issue JWT signed with JWT_SECRET; expiry 7 days.

REQ-AUTH-3: Enforce role-based access in feature routers; deny 403 on violations.

REQ-AUTH-4: On 401, client clears session and redirects to Login.

REQ-AUTH-5: Rate-limit auth endpoints to mitigate abuse.

4.2 Courses Management

Description and Priority:

Create/update/delete courses; enroll/drop; view catalogs and details (High).

Stimulus/Response Sequences:

Teacher creates course → course listed → students can enroll.

Student enrolls → course appears in “My Courses”.

Functional Requirements:

REQ-COURSE-1: CRUD operations on courses via /api/courses.

REQ-COURSE-2: Enrollment/drop endpoints update user-course relations.

REQ-COURSE-3: Return paginated lists with filters (TBD exact filters).

REQ-COURSE-4: Serve course assets from /uploads where applicable.

4.3 Assignments & Submissions

Description and Priority:

Teachers assign work; students submit; teachers grade; feedback loop (High).

Stimulus/Response Sequences:

Teacher creates assignment → student submits → teacher grades → student notified.

Functional Requirements:

REQ-ASSIGN-1: CRUD assignments via /API /assignments.

REQ-ASSIGN-2: Accept submissions (text/files) with validation and storage.

REQ-ASSIGN-3: Grade submissions; persist scores and comments.

REQ-ASSIGN-4: Emit assignment-graded event to concerned student(s).

4.4 Quizzes

Description and Priority:

Create, take, and score quizzes; track attempts (High).

Stimulus/Response Sequences:

Student starts quiz → answers → submits → receives results.

Functional Requirements:

REQ-QUIZ-1: CRUD quizzes via /api/quizzes.

REQ-QUIZ-2: Record answers and submissions; compute results.

REQ-QUIZ-3: Emit quiz-available when new quiz is published.

REQ-QUIZ-4: Secure timing and attempt limits (TBD specifics).

4.5 Announcements

Description and Priority: Broadcast course or system-wide updates (Medium).

Stimulus/Response Sequences:

Admin/teacher posts announcement → subscribed users receive real-time toast.

Functional Requirements:

REQ-ANN-1: CRUD announcements via /api/announcements.

REQ-ANN-2: Emit new-announcement to relevant rooms.

REQ-ANN-3: Display announcement list with read status.

4.6 Attendance

Description and Priority:

Track session attendance; start/stop sessions; record presence (Medium).

Stimulus/Response Sequences:

Teacher starts session → students check in → analytics update.

Functional Requirements:

REQ-ATT-1: Manage sessions and attendance records via /api/attendance.

REQ-ATT-2: Emit new-attendance-session events to enrolled students.

REQ-ATT-3: Role-based restrictions (teachers/admin only to start/close).

4.7 Gamification

Description and Priority: XP, levels, badges, rewards, leaderboard (Medium).

Stimulus/Response Sequences:

User completes action → gains XP → possible level-up/badge → leaderboard updates.

Functional Requirements:

REQ-GAM-1: Provide stats, achievements, rewards via /api/gamification/*.

REQ-GAM-2: Emit xp-gained, level-up, badge-earned events to users.

REQ-GAM-3: Enforce redemption rules and rate limits.

4.8 Analytics

Description and Priority: Overview metrics for admins/teachers (Medium).

Stimulus/Response Sequences:

Admin opens Analytics → backend aggregates → charts render.

Functional Requirements:

REQ-ANL-1: Serve analytics endpoints via /api/analytics.

REQ-ANL-2: Respect role permissions; anonymize sensitive data where needed (TBD).

4.9 Profile & Settings

Description and Priority: Manage personal info, avatar, preferences (Medium).

Stimulus/Response Sequences:

User updates profile → validation → persist → UI refresh.

Functional Requirements:

REQ-PROF-1: CRUD user profiles via /api/users.

REQ-PROF-2: Avatar uploads via /api/users/:id/avatar stored under /uploads.

REQ-PROF-3: Enforce ownership and access control.

4.10 AI Assistant

Description and Priority: Contextual assistance via Hugging Face APIs (Medium).

Stimulus/Response Sequences:

User prompts assistant → backend calls inference → response displayed.

Functional Requirements:

REQ-AI-1: Integrate Hugging Face with HUGGINGFACE_API_KEY via /api/ai.

REQ-AI-2: Rate-limit and sanitize inputs; redact secrets.

REQ-AI-3: Provide fallbacks on API errors and timeouts.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Navigation between modules shall occur with **minimal latency** to ensure smooth user experience for Students, Teachers, and Admin users.

- Data-driven views such as leaderboards, attendance tables, course progress lists, and reward histories shall load efficiently without noticeable delay.
- Concurrent access by multiple user roles (students, teachers, administrators) shall not degrade system responsiveness.

5.2 Safety Requirements

- Role-based access control shall ensure that **students, teachers, and administrators** can only access functionalities relevant to their roles, as reflected in distinct dashboards.
- Administrative operations such as **student management, teacher management, course progress tracking, and attendance & grade management** shall be restricted to authorized users only.
- Academic records including attendance status, grades, quiz scores, assignments, and certificates shall be protected against unauthorized modification.

5.3 Security Requirements

- The system shall implement **role-based access control** to ensure that Students, Teachers, and Administrators can access only their respective dashboards and functionalities.
- User authentication shall be required to access the platform, ensuring that only authorized users can view or manage academic data.
- Sensitive academic information such as **attendance records, grades, assignments, quiz scores, rewards, and course progress** shall be protected from unauthorized access.
- Administrative features including **student management, teacher management, course progress tracking, and attendance & grades management** shall be accessible only to authorized admin users.
- Certificate verification shall be performed using a **secure hash-based validation mechanism**, allowing authenticity checks without exposing internal system data.

5.4 Software Quality Attributes

- **Usability:** The system shall provide a clean, intuitive, and consistent user interface across all modules with clear navigation and readable layouts.
- **Reliability:** The platform shall consistently maintain accurate academic data such as course enrollments, submissions, XP points, attendance records, and certificates.
- **Maintainability:** Modular separation of student, teacher, and admin functionalities shall allow ease of maintenance and future updates.

5.5 Business Rules

- Users shall be classified into **Student, Teacher, and Admin** roles, each with predefined permissions.
- XP points and levels shall be awarded based on visible academic activities such as assignment submissions and quizzes.
- Course enrollment status shall determine access to related assignments, quizzes, and attendance records.
- Certificates shall only be issued upon successful completion of courses or achievements and stored with blockchain verification.

6. Other Requirements

- The system shall support **certificate verification** through a publicly verifiable hash mechanism.
- All dashboards shall display **real-time academic summaries** such as XP, progress levels, enrolled courses, assignments, quizzes, and attendance.
- The platform shall maintain consistent branding and layout across login, dashboards, and feature modules.
- The application shall function reliably within a browser-based environment as demonstrated by the local deployment.

Appendix A: Glossary **N.A.**

Appendix B: Analysis Models **N.A.**

Appendix C: To Be Determined List **N.A.**

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				